

SE3090 – Software Engineering Frameworks

Lecture 06 • Agentic Software Development 2

Agentic RAG

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@slit.lk | Course web: courseweb.slit.lk

Today's Roadmap – 3 Hours

Block A

0–35 mins

The grounding problem & the RAG pipeline

Why models hallucinate, fine-tuning vs RAG, loading, splitting and overlap

Block B

35–85 mins

Embeddings, vector search & RAG quality

Semantic search, similarity metrics, vector stores and retrieval evaluation

Block C

95–140 mins

Production-ready RAG

Query transformation, hybrid retrieval, reranking, evaluation and security

Block D

145–180 mins

Memory & the idea of tools

Short-term and long-term memory, conversational RAG, tools versus retrieval

Short breaks at 85–95 min and 140–145 min • Questions welcome at any point

PART 1 OF 4

The Grounding Problem & the RAG Pipeline

- The grounding problem: why models invent answers
- Fine-tuning vs RAG — when to use each
- The two-phase pipeline: indexing and querying
- Loading, splitting, and why overlap matters



The grounding problem

Ask a base model about **your** company's refund policy. It will **hallucinate** a plausible answer or say it doesn't know — it was never trained on your PDF, and it has no live access.

Fine-tuning

Idea: Re-train or adjust the model using domain-specific data.

Trade-offs: Costly, slow, **static**, Less transparent

RAG

Idea: Retrieve relevant information and include it in the model's prompt.

Trade-offs: Cheap, **live**, citeable ✓

RAG changes the input, not the model. That's why it's the default for "answer from my documents."

Fine-tuning vs RAG — when each?

Use fine-tuning when...

- You need a consistent **style, format, or behaviour**
- The "knowledge" is a **skill**, not a fact set
- You can afford to retrain on updates
- Teach the AI **how to answer** professionally.

Use RAG when...

- You need **current or private facts**
- The knowledge **changes** over time
- You need **citations / auditability**
- Give the AI **the correct information** before answering.

Often you use **both**: fine-tune the *behaviour*, RAG the *facts*.

Heuristic: **RAG** for what it should *know*, fine-tuning for how it should *act*.

RAG as a mental model: two phases

Phase 1: INDEXING (done once, offline)

docs → load → split → embed → **store in vector DB**

Phase 2: QUERYING (per question, online)

question → embed → retrieve top-k chunks → put chunks in prompt → model → **answer (+ citations)**

Hold these **two phases** separately.

Most RAG confusion comes from mixing "prepare the knowledge" with "answer a question."

Step 1 — Loading (get data in)

First , collect data from Any source — PDF, web page, database, Markdown, Slack export.

Convert every sources into a common **Document** format.

A **document** contains: **content** (The actual text.) + **metadata** (source, page, url, author, date...).

- One **loader interface**, hundreds of source types.
- **Metadata is precious** — it's helps with **filtering** and providing **citations**.

Design point: Always keep the content and its metadata together.

You can't cite a source you threw away.

Step 2 — Splitting (chunking)

We break documents into **passages** because:

- **Focused pieces:** Retrieval works best on focused pieces, not whole books.
- **Token budget:** Chunks must fit the budget alongside question and answer.

The craft: *chunks big enough to hold a complete idea, small enough to be precise.*

Too big

Retrieval returns unnecessary information, wastes tokens and makes the correct answer difficult to find.

Too small

Ideas are split across chunks, key context is lost, and retrieval misses them.

Chunk size and overlap are two important factors that affect RAG quality.

We'll tune it in the lab.

Choosing the Right Chunk Size

A good chunk keeps **one complete idea** together:

- **Keep related text:** Keep sentences about the same topic together.
- **Stay focused:** Avoid mixing several unrelated topics in one chunk.

The goal: *enough context to answer, with little unnecessary text.*

Start here

Try `chunk_size = 1000` characters.

Use `chunk_overlap = 100–200` characters.

Test and adjust

Ask real questions. Check the retrieved chunk, then increase or decrease the size.

There is no universal best chunk size.

Start with a sensible value, then test it using your own documents and questions.

Why Overlap Matters

Sometimes important information is divided between two chunks.

Chunk A ends here ↴
... annual plans is 30 | days. Monthly...
└─ Chunk B starts here

Without overlap, "30" and "days" land in completely different chunks.

Then the system may fail to understand the complete meaning: "30 days."

The Search Failure

A user query for "how many days for a refund?" may match neither chunk well, as the key semantic connection is completely broken.

The Solution: Overlap

Overlap repeats a small amount of text between neighbouring chunks. By keeping "30 days" together in both chunks, the context remains fully intact.

Trade-off: Safety vs. Efficiency

More overlap guarantees better boundary safety but generates more chunks, increasing storage costs and token consumption.

Typical overlap is ≈ 10–20% of chunk size.



Discussion Checkpoint #1 (5 min)

1. Your RAG bot gives a vague, half-right answer about pricing. Before blaming the model, **where do you look first**, and why?

2. You set **chunk_size = 50** characters to be "precise." What likely goes wrong?

With a neighbour — 5 minutes — then we discuss answers together.



Discussion Checkpoint #1 — Model Answers

Scenario 1: Vague Pricing Answers

Your RAG bot gives a vague, half-right answer about pricing. Before blaming the model, **where do you look first**, and why?

Checkpoint 1 Answer

Inspect the retrieved chunks first. Most failures in RAG are retrieval issues, not generation. Always audit the raw context fed to the model.

Scenario 2: Ultra-Precise Chunking

You set **chunk_size = 50** characters to be ultra-precise. What likely goes wrong?

Checkpoint 2 Answer

50 characters shreds ideas. Retrieval returns micro-fragments with no context. The model cannot answer from "confetti".

With a neighbour — 5 minutes — then we discuss answers together.

PART 2 OF 4

Embedding's, Vector search & RAG quality

- Keyword search vs semantic search
- Embeddings and how similarity is measured
- Vector stores and approximate nearest neighbours
- Evaluating retrieval and tuning RAG quality



The search problem

How do we find the chunks "relevant" to a question?

Keyword search

Matches **letters/words** directly.

For example, searching for "**car**" won't match "**automobile.**"

It can miss synonyms, paraphrases and differently worded questions.

Keyword search is still useful for exact values such as error codes, product IDs and names.

Semantic search

Matches by conceptual **meaning**.

For example, Therefore "**I forgot my login**" can match a passage about "**password reset**"

Finds relevant text even when the words are different.

Semantic search is useful when users express the same idea using different words.

Embeddings make semantic search possible.

Strong RAG often combines both methods: keyword search for exact terms and semantic search for meaning.

Embedding's — Turning Meaning into Numbers

An embedding converts text into a vector—a numerical representation of its meaning.

Vector Closeness

Similar meaning → nearby vectors

Different meanings → distant vectors

"reset my password" and
"I forgot my login" = **close**

"reset my password" and
"pizza toppings" = **far**

🔍 How RAG Uses Embedding's

1. Convert the user's question into a vector.
2. Compare it with stored chunk vectors
3. Retrieve the nearest and most relevant chunks.

The mathematical distance or angle between vectors directly measures **semantic similarity**.

Embeddings help RAG find relevant information even when the question and document use different words.

Why Are Embeddings High-Dimensional?

Embeddings use many values together to represent subtle differences in meaning.

More Dimensions = More Detail

An embedding may contain hundreds or thousands of values. Together, they represent topic, tone, context, and entities.

More dimensions help separate subtle meanings. **One value alone is not meaningful**; the full pattern carries the meaning.

Compare with Cosine Similarity

First, convert both texts into vectors. Then compare their **direction** using cosine similarity.

A **higher similarity** means closer meaning; a lower score means different meaning.

*High dimensions provide space to **represent detailed meaning**; cosine similarity helps RAG find the closest information.*

Measuring Closeness — Three metrics

Here, **a** and **b** represent two embedding vectors.

1. Dot Product

$$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$$

Multiply matching values and add them together.

A larger result can indicate greater similarity.

However, vector size or magnitude affects the result.

2. Euclidean Distance

$$||\mathbf{a} - \mathbf{b}|| = \sqrt{\sum (a_i - b_i)^2}$$

Measures the straight-line distance between two vectors.

Smaller distance means greater similarity.

It is also sensitive to vector scale.

3. Cosine Similarity

$$\cos \theta = \mathbf{a} \cdot \mathbf{b} / (||\mathbf{a}|| ||\mathbf{b}||)$$

Compares the direction or angle of two vectors.

It ignores vector magnitude.

Therefore, it is commonly used with text embedding's.

A Cosine score : $\cos \theta = 1$ means highly similar, 0 means unrelated, and -1 (negative) means different directions.

Example: “Forgot my password” and “Reset my login” should have a higher cosine similarity than “Forgot my password” and “Pizza toppings.”

L2 Normalization — Make Every Vector Length 1

If vectors are L2-normalized ($||a|| = ||b|| = 1$), then all three metrics are mathematically linked:

The Mathematical Equivalence

L2 normalization changes a vector so its total length becomes 1, without changing its direction.

$$||a - b||_2^2 = 2 - 2(a \cdot b) = 2 - 2 \cos \theta$$

On L2-normalized vectors, the squared Euclidean distance is directly proportional to the dot product and the cosine similarity.

After Normalization

- **Same score:** dot product and cosine similarity give the same ranking. Euclidean distance is also mathematically connected to them.
- **Same ranking:** Euclidean distance finds the same nearest vectors; smaller distance means closer.

Normalizing makes "similarity" about meaning, not text length or vector scale.

In Practical: **Normalize** document embedding's when storing them, **normalize each query once**, and use a fast dot product to find the most similar chunks.

How Embedding's Learn Meaning

Embedding models are trained with **contrastive learning** (a dual encoder) to map meaning to distance: Contrastive learning **pulls** related text **closer** and **pushes** unrelated text farther **apart**

The model receives

$$\mathcal{L} = -\log \left[\frac{e^{(\text{sim}(q,d^+)/\tau)}}{\sum e^{(\text{sim}(q,d)/\tau)}} \right]$$

Anchor (a): the query

Positive (p): the correct or related passage

Negative (n): unrelated passages

- **Query:** "I forgot my login"
- **Positive:** "Reset your password" • **Negative:** "Best pizza toppings"

The Dual Encoder Mechanism

A dual encoder converts the query and passage into separate vectors

- **Pull Related Pairs:** Training actively pulls the embedding's of related queries and positive passages closer together.
- **Push Negatives Apart:** Simultaneously pushes unrelated pairs far apart in the vector space.

Why it works: Similar meanings become nearby vectors—so "I forgot my login" ends up near "reset your password." **Distance now represents meaning.**

Searching fast: exact vs approximate NN

NN means Nearest Neighbor —finding the vectors most similar to the query.

Brute Force (Exact)

$O(n \cdot d)$

compares the query with every stored vector.

It gives the exact result.

However, it becomes slow with millions of vectors.

IVF Index

Inverted File

Groups similar vectors into clusters and searches only the nearest clusters.

HNSW Graph

$\sim O(\log n)$ search

creates a graph and quickly moves through connected vectors to find close matches.

Multi-layer graph of navigable small worlds.

PQ Compression

Product Quantization

Compress vectors into compact codes. Uses far less memory and boosts speed.

Managed Search: Vector databases like Pinecone, pgvector, and FAISS implement these complex indices. You get fast nearest-neighbour searches out of the box without building algorithms from scratch.

MMR — Relevant Results Without Repetition

Plain top-k can return near-duplicates. **Maximal Marginal Relevance** MMR re-ranks retrieved chunks to **balance relevance** and diversity.

How MMR Scores Each Chunk

$$\text{Score} = \lambda \times \text{relevance} - (1-\lambda) \times \text{repetition}$$

- **Relevance:** How well does this chunk match the query?
- **Repetition:** How similar is it to chunks already selected?

What λ (Lambda) Controls

- **λ close to 1:** Prioritize the most relevant chunks.
- **λ around 0.5:** Balance relevance with new information.
- **Lower λ** prioritizes diverse, less repetitive chunks.

Example: Plain top-k may return three password-reset chunks. MMR can return password reset, account recovery, and login security—more useful context for the LLM.

Evaluating retrieval — the metrics

How do you know retrieval is *good*? Score it against known-relevant passages.

@k means we evaluate only the first k results, such as the top 5.

1. Recall@k

(relevant top-k) /
(all relevant)

Captures:

Out of all the relevant chunks,
how many did we retrieve?

2. Precision@k

(relevant top-k) /
k

Captures:

Out of the retrieved chunks,
how many are actually
relevant?

3. MRR

mean of 1/rank of
first hit

Captures:

How high is the first correct
result ranked?

4. nDCG@k

DCG / IDCG

$DCG = \sum rel_i / \log_2(i+1)$

Captures:

Are the most useful results
placed near the top?

For RAG, Recall@k matters most — if the right chunk isn't retrieved, the model **cannot** answer correctly, no matter how good it is.

Vector stores — searching the galaxy

A **vector store** indexes embeddings so we can find nearest neighbours *fast* (even over millions of chunks).

Core Responsibilities

- A **vector store keeps**: The **chunk's embedding** + its **text** + its **metadata**.
- Given a query embedding, return the **top-k** closest chunks dynamically.

Where it Runs

Local / In-Process:

FAISS, Chroma

Managed / Production:

Pinecone, pgvector, Weaviate

The store is an implementation detail. The **concept** is "approximate nearest-neighbour search over meaning."

Assembling the answer — "stuffing" context

The simplest, most common RAG strategy: retrieve, insert, and instruct. It means placing all selected document chunks directly into the LLM prompt.

The process has three steps:

- 1. Retrieve:** Find the top-k relevant document chunks dynamically.
- 2. Stuff:** Insert them into the model's prompt *as rich context*.
- 3. Instruct:** Tell the model to answer strictly using only that context.

Anti-Hallucination - Guardrail

The final instruction forms the system's core defense:

"Answer only from this context; if it's not here, say you don't know."

This changes the task from risky recall to reliable reading comprehension.

The model's job shifts from recall (risky) to reading comprehension over supplied text (reliable). That shift is RAG.

Grounding & citations

Trustworthy AI isn't just "right" — it **shows its work**.

Verifiable Metadata

Because each chunk carries **metadata**, the model can say *where* an answer came from:

"Refunds take 5–7 days [handbook, Billing]."

Structured Outputs

We can also return the result as a **structured output** containing

answer + sources + confidence

Citations turn an opinion into a verifiable claim, allowing users and auditors to trace information back to its source.

What makes RAG good or bad — the retrieval

RAG quality is **mostly retrieval quality**. The dials:

The Retrieval Levers

Chunk size / overlap

Smaller chunks are more focused. **Larger chunks** provide more context. **Overlap** prevents information from being split between chunks.

k (chunks retrieved)

Controls how many chunks are retrieved. A larger k may improve recall, but it can also add noise and increase token usage.

Diversity (MMR)

Reduces duplicate results and provides more diverse information.

Filtering & Representation

Metadata filtering

Limits the search by document, section, date or category.

Embedding model

A better model can represent meaning more accurately and improve matching.

Debugging maxim: "The LLM is wrong" is usually "retrieval gave it junk." Inspect the retrieved chunks *first*.

RAG failure modes (and the fix)

Symptoms map to causes — and **almost every cause is retrieval**:

SYMPTOM	LIKELY CAUSE	THE FIX
Vague / generic answer	Chunks too big or irrelevant	Tune chunk size; raise relevance
"I don't know" when it should know	Retrieval missed it	Smaller chunks, higher k, better query
Confident but wrong	Model used memory, not context	Strengthen the "only from context" guardrail
Repetitive context	Near-duplicate chunks	Use MMR for diversity
Slow / expensive	k too high, chunks too big	Lower k; right-size chunks

Almost every row is retrieval, not the model. Internalise that.

PART 3 OF 4

Production-ready RAG

- From demo RAG to production RAG
- Query transformation, hybrid retrieval and reranking
- Pipeline patterns: routing, multi-hop and self-correction
- Evaluation, freshness, cost and security



From demo RAG to production RAG

A **basic RAG demo** usually uses one user, one document and simple questions.

Production RAG is the basic RAG pipeline strengthened for real users, real data and real risks:

Demo assumption

- One user, one PDF
- Clean, self-contained questions
- Documents never change
- "Looks right" is enough
- Everyone may read everything

Production reality

- Millions of chunks, many users, permissions
- Vague, conversational, multi-part queries
- Docs update daily → **stale index**
- Must **measure** quality, cost, latency
- Access control, PII, prompt injection

Production RAG still has the same main phases:

1. **Ingestion:** Prepare, chunk, embed and index documents.
2. **Query time:** Retrieve relevant chunks and generate the answer.

The RAG Upgrade Map

Improvements are required on both the query side and the index side.

Query side – happens when the user asks a question:

Understand: Rewrite unclear queries or create multiple query

Retrieve: Combine semantic vector search with keyword search

Filter: Apply metadata filters and access-control rules.

Refine: Rerank, remove weak results and compress unnecessary content.

HyDE creates a possible answer first and uses it to search for similar documents.

ACL ensures that users retrieve only the documents they are permitted to access.

Index side – happens when preparing the documents:

- **Split documents** using smarter or semantic chunking.
- **Use an embedding** model suitable for the domain.
- **Store embeddings** together with text and metadata.
- **Update the index** when documents change and maintain versions.

Topology: routing · multi-hop · self-correcting · agentic composition

Around It: evaluation · caching · tracing · user feedback · security guardrails

Fix the question first — query transformations

Users don't ask "retrievable" questions. Transform the query **before** embedding it:

Rewrite / condense

- Converts a follow-up question into a complete standalone question.
- Example: “Is it cheaper?” becomes “Is Pinecone cheaper than Weaviate?”

Multi-query

- Generates several versions of the same question.
- Retrieves results for every version and combines them.
- This improves recall when documents use different wording.

Step-back

- Creates a more general version of an overly specific question.
- This helps retrieve broader background information.

HyDE (Hypothetical Document Embeddings)

- The LLM first generates a possible or hypothetical answer.
- That answer is embedded and used to search for similar documents.

LangChain: Improve the question before searching to improve the retrieved context.

Hybrid Retrieval — Meaning(dense) + Exact Words(keyword)

Embeddings miss **exact strings** (error codes, SKUs, names, acronyms); keyword search (**BM25**) misses **paraphrases**. Production systems run **both** and fuse rankings.

Reciprocal Rank Fusion (RRF)

RRF Formula:

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} 1 / (k + \text{rank}_r(d))$$

- A document ranked high by **either** ranker scores well.
- Parameter **k** (≈ 60) damps the long tail effect.
- Strictly rank-based: **no score normalization** is needed.

Implementation & Frameworks

LangChain Integration:

```
EnsembleRetriever(  
    retrievers=[bm25, vector],  
    weights=[0.4, 0.6]  
)
```

- Combines sparse and dense retrievers seamlessly.
- Configurable weights for fine-tuned hybrid fusion.

Rule of thumb: Hybrid retrieval consistently beats either model alone — especially on **IDs, jargon, and names**.

Hybrid Retrieval — Meaning(dense) + Exact Words(keyword)

combines vector search with keyword search. Vector search understands meaning; BM25 catches exact terms. RRF combines both ranked lists.

1 & 2 — Search in Two Ways

1. Vector Search (Dense)

- Search by Finds meaning, synonyms and paraphrases
- May miss exact IDs, codes, names and SKUs.

2. Keyword search or BM25:

- May miss synonyms or different wording.

How They Work Together

Three Simple Steps

1. Run vector search
2. Run BM25 search
3. Merge both rankings with RRF
 - RRF gives more points to results ranked near the top.
 - It uses ranks, so raw scores do not need normalization.

Example: Query “ERR-401 login issue” → BM25 finds the exact code; vector search finds **authentication-failure documents**. The combined list contains both.

Retrieve wide, Re-rank narrow

Production RAG commonly uses a **two-stage retrieval process**.

Stage 1: Hybrid / Vector Search (Wide)

Retrieves the top ~50 candidates. Fast and approximate to ensure high recall.

Stage 2: Cross-Encoder Re-ranker (Narrow)

Filters down to the best 3–5 documents. Slow but highly precise context for the prompt.

A **bi-encoder** creates the query and document embedding's separately.

- Comparison uses a fast dot product.
- It is suitable for searching millions of documents.

A **cross-encoder** reads the query and document together.

- It provides more accurate relevance scores.
- However, it requires a model call for every query-document pair, so it is slower and more expensive.

Cheap-then-precise beats precise-everywhere. LangChain: `ContextualCompressionRetriever` wrapping a reranker (e.g. Cohere Rerank, BGE cross-encoder).

Smarter indexing — beyond flat chunks

Basic RAG often splits documents after a fixed number of characters.

This may break sentences, sections or code blocks and remove important context.

Smarter indexing creates chunks based on the document's meaning and structure.

Structure & Parents

- **Structure-aware splitting:**

split using headings, paragraphs, tables or code blocks. This keeps related information together.

- **Parent-document retrieval:**

Search using small, focused chunks. When a small chunk matches, return its larger parent section. This combines accurate matching with enough surrounding context.

Summaries & Context

- **Multi-vector / summaries indexing:**

Create summaries or possible questions for each chunk. Search using these additional representations. Return the original document content to the LLM

- **Contextual chunk headers:**

Add the document title, section name or short description before embedding. This helps the embedding model understand what the chunk is about.

LangChain integration: `ParentDocumentRetriever` · `MultiVectorRetriever`

Measured gains: Better document preparation can improve retrieval without changing the LLM.

After retrieval, before the prompt

Retrieval may return useful chunks together with weak, repeated or unnecessary content. Before sending them to the LLM, we should clean and organize the context.

Filtering & Compression

- **Score threshold:**

Remove chunks with a relevance score below a selected threshold. If every result is weak, the **system should say** “I don’t know” instead of using unrelated information.

- **Contextual compression:**

Remove irrelevant sentences from retrieved chunks. Keep only the parts that help answer the user’s question. This reduces token usage and noise.

Ordering & Deduplication

- **Lost in the middle:**

LLMs may pay more attention to information at the beginning and end of a long prompt. Place the strongest chunks in important positions. LangChain provides **LongContextReorder** for this purpose.

- **Deduplicate:**

Remove chunks that contain almost identical information. MMR can help retrieve more diverse chunks.

The Gold Standard: The prompt's token budget is prime real estate — spend it strictly on **signal**, not noise.

Reference: Liu et al. (2023) "Lost in the Middle: How Language Models Use Long Contexts"

Evaluating RAG end-to-end — the triad

End-to-end RAG evaluation checks the relationship between **three elements**: The user's **question**, The **retrieved context**, The **generated answer**:

The Core Triad Metrics

Faithfulness: Is every statement in the answer supported by the retrieved context? Unsupported claims indicate hallucination

Context Relevance: Does the retrieved context contain useful information for answering the question? Irrelevant chunks indicate a retrieval problem.

Answer Relevance: Does the final answer directly address what the user asked? An answer may be factually correct but still be off-topic.

Continuous Quality (CI/CD)

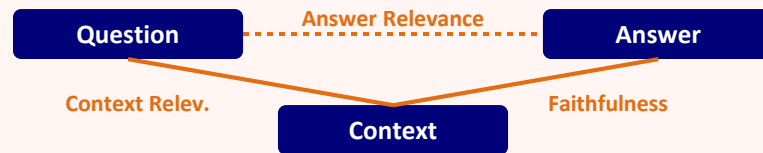
Golden Set: Create a golden test set containing sample questions, expected answers and correct source documents.

CI for Quality: Run this test set whenever chunking, embeddings, retrieval settings or prompts are changed.

LLM-as-Judge: Use Frameworks such as Ragas and LangSmith can help automate evaluation using an LLM as a judge.

Evaluation Mantra

"If you can't measure it, you can't tune it." A system that merely "feels better" will not survive in production.



RAG Architectures — the maturity ladder

How the research literature classifies RAG systems (Gao et al., 2023 — the RAG survey): three levels of RAG system maturity.

Naive RAG

Shape:

Uses a simple linear pipeline:
index → retrieve → generate

Adds:

The basic laboratory pipeline

Example : Simple FAQ chatbot

Advanced RAG

Shape:

Keeps the main pipeline but improves its stages.

+ Pre-retrieval & post-retrieval stages

Adds:

Transforms, hybrid, re-ranking, context compression

Example : Enterprise document assistant

Modular RAG

Shape:

Changes the pipeline into a collection of reusable modules. Different queries can follow different paths. Swappable modules, **non-linear** flows

Adds:

Routing, loops, memory, external tool use

Example : Complex assistant using documents, databases and tools

Key Difference: Everything up to Advanced RAG upgraded the **stages**, but **Modular RAG** changes the **shape**—the pipeline becomes a **graph** of modules composed per use case.

Core Rule: An architecture = **which modules** + **how data flows** between them.

Routing & Branching — Choose the Right Path

The router uses structured output to choose a source, tool, or skip-retrieval path.

How the Router Chooses a Path

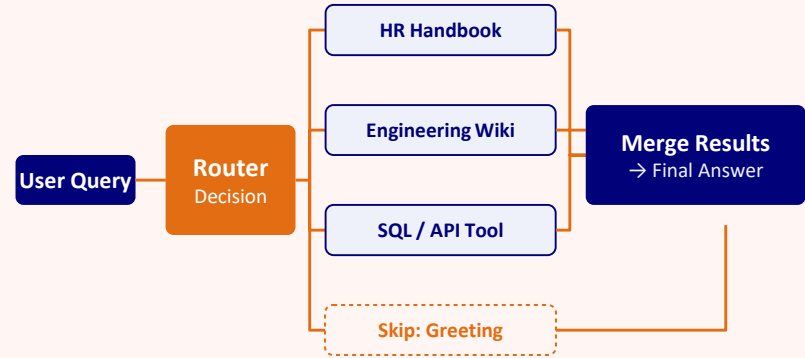
- 1. Route** — first examines the question and choose the correct source, retriever, or tool.
- 2. Decompose** — Split a complex question into smaller sub-questions. Route each sub-question to the correct source. Retrieve the answers and merge them.

Ex. “What is my leave balance, and how do I deploy the API?”

Leave balance → HR or employee database

API deployment → Engineering Wiki

- 3. Skip-retrieval branch:** —Simple greetings do not need document retrieval.
- 4. Merge + Re-rank :** — After retrieval, results can be **merged** and **re-ranked** before generating the final answer.



Key Idea : Routing uses an LLM’s structured output to control which path the RAG system follows.

Pipeline Patterns — Iterative & Self-correcting

Basic linear RAG retrieves information only once.

Complex questions may require the system to search, inspect the results and search again.

Multi-hop

Retrieve information for the first part of the question.

Read the result and create the next query.

Continue until all required information is collected.

Corrective RAG

First evaluate the quality of the retrieved documents.

If the documents are weak, rewrite the query or search another source, such as the web.

Self-RAG

The model checks its own draft answer.

It asks: Is the answer supported? Is the context relevant? Do I need more information?

If necessary, it retrieves again.

GraphRAG

Organizes entities and their relationships as a knowledge graph.

Useful for questions involving connections, trends or information across many documents.

Key Takeaway: When a RAG pipeline decides whether to retrieve again, it begins to behave like an agent.

Agentic RAG — choosing an architecture

Start with simple RAG. Add routing, loops or tools only when evaluation shows a clear failure.

Agentic RAG

An agent can decide:

- **Should** I retrieve information?
- **Which** source or tool should I use?
- Do I need to search again?
- Is the evidence strong enough to answer?
- When should I stop?

Observed Problem	Recommended Pattern
Multi-part question is only partly answered	Decomposition / RAG-Fusion
Correct information exists in another source	Routing
Answer requires several connected facts	Multi-hop / Iterative RAG
Retrieved evidence is weak or conflicting	Corrective RAG + Thresholds
"Question asks for a whole-corpus summary or trend	GraphRAG / Hierarchical Summaries

Start simple → Evaluate → Identify the failure → Add the smallest useful pattern → Test again.

Track **answer quality, latency and cost** after every architectural change.

Key message: Complexity is justified only when it solves a measured problem.

Keeping RAG Reliable — Freshness, Cost & Speed

A RAG system needs continuous maintenance after deployment.

Keep the Index Fresh

- Embed only new or updated documents.
- Remove deleted information.
- Run synchronization regularly.

Changing the Embedding Model

- Old and new embedding's cannot be mixed.
- Create a new index → re-embed everything → test → switch safely.

Control Cost

- Cache existing embedding's.
- Do not reprocess unchanged documents.
- Process document updates in batches.

Reduce Latency

- Measure each step: rewrite → retrieve → rerank → generate.
- Run independent searches in parallel.
- Stream the answer to the user.

Key Takeaway: Production RAG is a continuous pipeline—synchronize, monitor, test and improve it regularly

RAG Security — Retrieved Content Can Be Unsafe

Documents retrieved for the LLM must be treated as untrusted input.

RAG Security

The **Retriever** as an Entry Point

Retriever = Input Channel

Anything stored in retrieved documents may enter the LLM prompt. Therefore, retrieved content should not be trusted automatically..

Risk	Defence
Indirect prompt injection - A document contains hidden instructions	Treat retrieved text as data , not instructions. Separate it clearly and validate the output.
Data leakage across users - Unauthorized data is retrieved	ACL-aware retrieval - Filter documents using the user's role and permissions.
PII in chunks - Personal data enters the prompt	Remove or mask PII before indexing. Audit what enters the prompt.
Citations as cover - A citation does not support the answer	Check that the cited passage directly supports the claim

Key Takeaway: Filter, restrict and validate retrieved content before allowing the LLM to use.



Discussion Checkpoint #2 (5 min)

1. Why must the **same embedding model** be used for indexing and querying?

2. A legal-search RAG returns 5 chunks that are basically the same paragraph. **Which lever fixes this**, and what does it trade away?

With a neighbour — 5 minutes — then we discuss answers together.

PART 4 OF 4

Memory & the idea of tools

- Why every model call is stateless
- Short-term memory: windows, summaries and budgets
- Long-term memory: semantic, episodic and procedural
- Conversational RAG and the idea of tools



Memory — the forgetting problem

LLM calls are stateless: Each request starts without the previous conversation. The application must provide the relevant history again.

Where Is Memory Stored?

- Stored by the application or a database.
- Use a **user ID**, **session ID** or **thread ID**.
- Keep each user's conversation separate.

How Much Should We Keep?

- Keep recent messages and important facts.
- **Summaries** or **remove** older messages.
- This reduces token cost, latency and context overflow.

How It Works: Load relevant history → Add the new message → Send both to the LLM → Save the response

Key Takeaway: The model does not remember by itself—the application stores and re-sends the relevant context.

Four Types of Agent Memory

Different memory types help an agent handle the current conversation, past events, known facts and instructions.

Memory Type	Simple Meaning	Example in an AI Agent
Working / Short-Term	<i>the current exchange</i>	Recent messages inside the context window
Episodic — events	<i>What happened before</i>	stored past sessions & interactions, summarised
Semantic — facts	<i>what I know</i>	User preferences, business facts and RAG knowledge
Procedural — skills	<i>how to act</i>	System prompts, business rules, workflows and tool instructions

Short-term memory is included directly in the prompt.

Long-term memory is stored outside the model and must be saved, searched and added back when relevant.

Short-Term memory — the loop

The LLM does *not remember automatically*. The application repeats a memory loop for every user message

State Management

Before each call:

- Read the user's session or thread ID.
- Load that session's conversation history.
- Combine: system instructions + history + new message.

After each call:

- Return the generated answer.
- Save the new message and answer to the same session.

The Loop (Turn N)

```
load history(session)
  ↳ [system + history + new msg]
    ↳ model → reply
save(new msg, reply) ←
```

Storage Options

- **In-memory:** Simple, but lost when the application restarts.
- **Redis:** Fast and useful across multiple servers.
- **Database:** Persistent and suitable for long-term storage.

Key Takeaway: Every turn must load from and save to the correct session.

Short-Term memory— strategies & trade-offs

Conversation history grows after every message. More history improves memory, but also increases tokens, **cost**, latency and **context-overflow** risk.

Strategy	How It Works	Trade-off - (Benefit and Limitation)
Keep Everything	Send the complete conversation every time	Remembers more details, but becomes expensive and may exceed the context limit
Sliding Windowing	Keep only the most recent N messages	Fast and affordable, but may forget earlier information
Summaries Older Messages	<i>Replace older messages with a short summary</i>	Keeps the main ideas, but may lose important details
Trim by tokens	Remove the oldest messages until the history fits	Keeps cost predictable, but may remove useful context

Recent messages + summary of older messages + important facts stored separately

Key Takeaway : Choose a strategy based on what the application must remember, how long it must remember it and the available cost.

Short-Term memory in LangChain

The lab pattern — wrap **any** chain; history is loaded/saved per **session_id**:

```
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

sessions: dict[str, ChatMessageHistory] = {}

def get_history(session_id: str) -> ChatMessageHistory:
    return sessions.setdefault(session_id, ChatMessageHistory())

chatbot = RunnableWithMessageHistory(chain, get_history,
    input_messages_key="input", history_messages_key="history")

chatbot.invoke({"input": "Hi, I'm Lasal!"},
    config={"configurable": {"session_id": "user-42"}})
```

Windowing & Budgets

Pipe `trim_messages(strategy="last", max_tokens=...)` in front of the prompt to avoid context overflow.

Durability

Swap the store seamlessly: use `SQLChatMessageHistory` (SQLite/Postgres) or Redis with **zero chain changes**.

RAG changes the input, not the model. *RunnableWithMessageHistory* handles state transition dynamically under the hood.

Long-term memory — remembering across sessions

Long-term memory allows an assistant to remember selected information after the current conversation ends.

Should We Avoid?

- Do not resend every past conversation.
- Do not insert months of raw chat logs into one prompt.
- This increases cost, latency and context-overflow risk.
- More history does not always produce a better answer.

Better Long-Term Memory Approach

- Identify useful facts, preferences or events.
- Store them outside the model using the correct user ID.
- Search for relevant memories when needed.
- Add only the selected memories to the new prompt.

Long-term memory = **select** useful information → **store** it outside the model → **retrieve** it when needed. *It follows the same pattern as RAG, but applies it to user facts, past events and behavior.*

Semantic Memory — remembering useful facts

Semantic memory stores stable facts about a user or the world—not complete conversations: **extract** → **store** → **retrieve** → **inject**

```
class UserFact(BaseModel): # Lecture structured output!
    fact: str # e.g. "prefers concise answers; codes in Python"
    category: Literal["preference", "biography", "project"]

extract = extract_prompt | model.with_structured_output(UserFact)
fact = extract.invoke({"turn": last_exchange}) # after turn or in bg

memory_vs.add_texts([fact.fact], metadatas=[{"user": uid, "cat": fact.category}])
memories = memory_vs.similarity_search(question, k=3, filter={"user": uid})

# → format into system prompt: "Known about this user: ..."
```

How Semantic Memory Works

1. **Extract and validate** : Identify a clear and useful fact from the conversation.
2. **Store** : Save it with the user ID, category, source and timestamp.
3. **Retrieve** : Search for facts related to the user's current question.
4. **Use and maintain** : Add relevant facts to the prompt. Update or delete outdated facts.

Memory Flow: Extract → Validate → Store → Retrieve → Use

Key Takeaway: A user profile is semantic memory about a person. RAG is semantic memory retrieved from documents.

Episodic Memory — remembering past events

*Episodic memory stores summaries of previous interactions: **what** happened, **when** it happened and **who** it involved.*

How Episodic Memory Works

1. **Capture the event** : Summarise an important interaction or completed session.
2. **Store it with metadata** : Save the user ID, timestamp, topic and outcome.
3. **Retrieve relevant events** : Search using relevance and recency, filtered by the correct user.
4. **Use the memory** : Add only the most relevant past events to the new prompt.

Implementation

```
# LLM-written summary & vector search
episode = summarise.invoke({"messages":
    history.messages})
memory_vs.add_texts([episode], metadatas=[{"user": uid,
    "ts": time.time()}])

past = memory_vs.similarity_search(question, k=2,
    filter={"user": uid})
```

Episodic memory is RAG over selected past experiences. Remember meaningful events—not complete transcripts.

Storing successful past interactions and replaying them as few-shot examples is episodic memory used for quality, not just personalization.

Putting it together — a memory architecture

*Agent memory combines a read path with a controlled write-back loop around the LLM.. Every arrow is a **Runnable** you already own.*

Before Answering - Read Path (Every Turn)

User Msg —► Short-Term Buffer —► Model —► Reply

1. **Episodic**: Relevant past episodes (similarity + recency)
2. **Semantic**: User facts / profile (structured vector store)
3. **Procedural**: System prompt + standing rules

After Answering = Write Path (After Turns / BG)

The loop captures and persists fresh context actively:

1. **Append** to chat history
2. **Extract** key structured facts
3. **Summarise** semantic episodes
4. **Reflect** on rules and behaviors

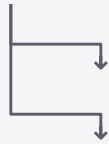
Composing retrievers, structured outputs, and summarisation chains. *Your existing runnable modules chain together to build complete, production-grade memory architectures.*

Conversational RAG — Memory + Retrieval

Memory understands the conversation; **retrieval** supplies trusted, up-to-date facts.

Previous Msg : “Tell me about the Team plan.”

Follow-up: “Is it cheaper yearly?”



1 **Memory:** “it” → Team plan; rewrite

2 **Retrieval:** Fetch current yearly pricing

3 **LLM:** Grounded, Context-Aware Answer

Who Does What? (Context + Facts)

- **Memory** — resolves “it” and rewrites: “Is the Team plan cheaper yearly?”
- **Retrieval** — finds trusted, current pricing.
- **LLM** — combines both into a grounded answer.

Another follow-up: “What about for 20 users?”
Memory identifies the plan; retrieval finds the price.

Memory tells the system **what the user means**; retrieval tells it what is currently true.

Tools vs RAG — two ways to ground

RAG provides reliable knowledge, while tools provide live data, calculations and controlled actions.

Comparison	RAG	Tools
What does it do?	Retrieves relevant content and adds it to the prompt	Calls a function, database or external API
Best for	Policies, manuals, reports and knowledge bases	Live data, calculations, system updates and actions
Example	“What is our refund policy?”	Check the status of order 5482.”
Result	Evidence-based answer with supporting context	Structured result or confirmed action

Use RAG when the answer is in documents. Use tools when the task requires current data, computation or action. Combine both for reliable agentic workflows.

Next Lecture & How to Prepare

Lecture 07 — Agents & Multi-Agent Systems: Automating the tool loop, coordinating multiple agents, and adding middleware, guardrails, & observability

- Work through the Lecture 06 lab sheet end to end
- Review the background reading on RAG, embeddings and agent memory
- Experiment with Azure AI Foundry or Google AI Studio before the next session